Спершу прибрати ручне введення, потім вимірювати

Ядро збудоване технічно акуратно, але користь не доведена жодним реальним прогоном. Головна перешкода на шляху до ефективності не в коді, а в тому, що найважчу роботу програми зараз робить людина: заповнює JSON приблизно з 20 полів на кожне оголошення, зокрема власноруч ставить оцінки відповідності.

15 жовтня 2026 р. Аналіз коду архіву 0.6.1, документації в ньому та плану ChatGPT від тієї ж дати. Друга думка, не заміна плану ChatGPT: більшість його знахідок я підтверджую, а розходимося в порядку робіт.

106
тестів пройшли, 63 с, Python 3.13

0
реальних прогонів на живих оголошеннях

~20
полів JSON людина заповнює на одне оголошення

~9
оголошень максимум у ліміті контексту 32 КБ (імпорт дозволяє 100)

Головна теза. Пілот із 10 задач, якщо запустити його на поточній версії, виміряє переважно час заповнення JSON. Звичайний ChatGPT («ось оголошення, чи варто відгукуватися?») майже напевно буде швидшим, і критерій «−20% активного часу» не виконається з відомої вже зараз причини. Тому одна коротка ітерація з полегшення введення має йти *до* пілота, а не після нього.

Цільовий щоденний шлях

1. Вставити посилання і текст оголошення, можна кілька → **2. Розбір моделлю** поля, бюджет, строк, вимоги, оцінка відповідності профілю → **3. Підтвердити 4–5 полів** форма, не JSON → **4. Картка рішення** відгукнутися / пропустити / уточнити + чернетка відгуку → **5. Записати результат** відгук → відповідь → виграш

Ефективність визначаю так само, як ChatGPT: якість не гірша, а активного часу власника менше. Але додаю другу вісь, **практичне використання**: якщо через два тижні власник не відкриває програму сам, без протоколу, значить вона не корисна, хоч би що показали заміри.

Оцінка плану ChatGPT

Загалом план сильний: чесно розмежовує доведене й недоведене, має добрі метрики та три корисні контрольні проби. Для двох його знахідок я окремо підтвердив механізм за кодом. Розходимося головним чином у пріоритетах.

ПУНКТ CHATGPT	МОЯ ПОЗИЦІЯ	КОМЕНТАР
Висновок агента не входить у звіт (P0)	ЗГОДЕН	Підтверджую: `render_report` будується лише з полів матеріалів, а `agent_runs` живуть окремо. Але це симптом глибшої проблеми: рішення *взагалі* не залежить від моделі (див. с. 3).
Часова нестабільність порівняння (P0)	ЗГОДЕН	Механізм підтверджую: `assess()` бере поточний час і входить у `projection`, від якої залежить `snapshot_hash` експерименту. Пробу на 49 годин сам не повторював. Пріоритет для мене P1: поки експерименти заморожені, на щоденну роботу це не впливає.
Ручна підготовка JSON (P1)	ПІДВИЩИТИ ДО P0	Це найбільший і вже відомий споживач часу. ChatGPT пропонує спершу виміряти, а потім вирішувати. Я пропоную виправити це до пілота, бо результат виміру передбачуваний.
Зв'язати витрати з `run_id`	ЗГОДЕН, P2	Корисно, але витрати на Ollama дорівнюють нулю, а для ChatGPT невідомі. На рішення «чи користуватися» це не впливає.
Класи помилок, технічний повтор, швидкий прогін на 16 оцінок	ВІДКЛАСТИ	Це вдосконалення підсистеми «Покращення», а вона заморожена. Із цього блоку зараз потрібні тільки дві речі: збій формату не має з'їдати квоту з 4 запусків, і має бути один автоматичний повтор.
Калібрування вибору моделі	ЗГОДЕН	Режим `auto` бере найменшу модель. Рекомендувати модель за обсягом пам'яті й запам'ятовувати вибір.
Резервна копія, відновлення, версія схеми	ЗГОДЕН	Потрібно до того, як у базі з'являться реальні дані: нумеровані міграції через `PRAGMA user_version`.
Екран «Нова / У роботі / Готове», майстер запуску	ЗГОДЕН	Додаю: розділи 04–06 («Покращення», «Досвід і клієнти», «Результат для клієнта») сховати за перемикачем «експериментальне».
Драбина автономності A0–A4	ЗГОДЕН	Структура добра. Пропоную лише стиснути A1, щоб він охоплював і розбір, і чернетку відгуку.

Чого в плані ChatGPT бракує

- **Оцінки `fit/scope/portfolio` вводить сама людина.** Тож бал 0–100 — це її власні числа, перемножені на ваги 0,5 / 0,3 / 0,2. Програма не оцінює відповідність замовлення.
- **Типовий вердикт — «Потрібно уточнити».** Шаблон за замовчуванням дає `needs_review` із 6 зауваженнями, і більшість із них про службові поля (`reviewer`, `source_opened`, `checked_at`), а не про суть замовлення. Користь від такого сигналу низька.
- **Чернетка відгуку замовнику** — найбільша окрема економія часу, а в плані її немає. Надсилати її людина й далі буде сама.
- **Перевірка цитат одночасно надто сувора і надто м'яка.** ChatGPT показав м'якість: неправдиве твердження з точною цитатою проходить. Але є і зворотний бік: одна неточна цитата (лапки «» замість "", тире) відкидає *всю* відповідь, а для Ollama ще й зараховується в ліміт 4 запусків.
- **Ліміти розходяться:** імпорт до 100 матеріалів, а в контекст агента вміщується приблизно 9.

Що обмежує ефективність

P0 — перед пілотом; P1 — у тій самій або наступній ітерації; P2 — за результатами вимірів. Пріоритет означає вплив на корисність, а не на безпеку.

ЧЕРГА	ЗНАХІДКА	ДОКАЗ	РІШЕННЯ
P0	Ручне введення ~20 полів JSON на оголошення	`data/materials-template.json`; `validate_item` вимагає структуровані поля, оцінки й підстави	Вставлення тексту й посилання → розбір моделлю → підтвердження 4–5 полів у формі
P0	Програма не оцінює відповідність	`rank_items`: `score` = fit·0,5 + scope·0,3 + portfolio·0,2 із чисел, введених людиною	Оцінку дає модель із поясненням і цитатами; людина може змінити, і цю зміну записуємо
P0	Рішення й аналіз моделі розірвані	Вердикт рахує тільки `assess()`; `agent_runs` у звіт не потрапляють (підтверджує ChatGPT)	Одна картка рішення: вердикт правил + висновок моделі + зауваження перевіряльника + чернетка відгуку
P0	Шум у вердиктах	Шаблон за замовчуванням → `needs_review`, 6 зауважень службового характеру	Розумні значення за замовчуванням: час перевірки = зараз, перевіряльник = власник, «джерело відкрито» = одна галочка. Попередження розділити на «блокує рішення» і «довідково»
P1	Перевірка цитат «все або нічого»	`validate_result` кидає виняток на першій невідповідності	Нормалізувати лапки, тире й регістр; невідповідні висновки відкидати поштучно з позначкою; один авто-повтор із текстом помилки; збої формату не рахувати в ліміт
P1	Ліміт контексту не узгоджений з імпортом	`MAX_INPUT_BYTES` = 32 КБ, ≈ 6 КБ базового запиту + ≈ 2,6 КБ на оголошення; імпорт дозволяє 100	Показувати, скільки оголошень вміщується; автоматично ділити добірку на частини
P1	Слабкий вибір моделі	`auto` → `models[0]`, тобто найменша модель	Рекомендація за обсягом пам'яті, запам'ятований вибір, попередження про модель, меншу за 7B
P1	Немає міграцій і відновлення	`CREATE TABLE IF NOT EXISTS` без версії схеми; експорт є, імпорту немає	`PRAGMA user_version`, копія бази перед оновленням, імпорт експорту
P1	Інтерфейс ширший за сценарій	6 розділів, 3 із них заморожені; 7 документів змішують російську й українську	Головний екран лише для відбору; документацію звести до README, CHANGELOG і протоколу пілота, однією мовою
P2	Часова нестабільність експериментів	Механізм підтверджено за кодом (див. с. 2)	Зафіксувати час оцінки в знімку, коли експерименти розморозять
P2	Опитування стану кожні 2 с повертає всю задачу	`static/app.js`: `setInterval` → `/api/tasks/:id` разом із журналом	Легкий ендпоінт статусу; оптимізувати після виміру

Що вже добре і чого не чіпати: сервер слухає лише локальну адресу й перевіряє токен та Origin; процеси моделей обмежені за часом і виводом; платні виклики автоматично не вмикаються; після перезапуску виклики не повторюються наосліп; транзакції SQLite коректні. Усе це стабільний фундамент: на ньому варто будувати, а не переписувати.

Етап 1 · версія 0.7 «вставив → рішення»

Орієнтовно 3–5 робочих днів. Мета: щоб пілот вимірював саму програму, а не ручне заповнення JSON.

РОБОТА	КОНКРЕТНЕ ПРИЙМАННЯ
1. Поле вставлення	Посилання + сирий текст одного або кількох оголошень. JSON-шаблон лишається запасним способом. Помилка розбору не губить вставлений текст.
2. Розбір моделлю	Модель повертає назву, бюджет із підставою, строк, статус, обов'язкові вимоги, `fit/scope/portfolio` із поясненням. Кожне поле має цитату з вставленого тексту, а поле без цитати показується як «невідомо».
3. Форма підтвердження	Людина бачить і змінює тільки 4–5 критичних полів: бюджет і хто його вказав, строк, статус, вимоги, оцінку відповідності. Змінені значення позначаються як «виправлено власником».
4. Значення за замовчуванням	Час перевірки = зараз, перевіряльник = власник профілю, «першоджерело відкрито» = галочка. Жоден із цих пунктів сам по собі не дає статусу `needs_review`.
5. Картка рішення	«Відгукнутися / Пропустити / Уточнити», три причини з цитатами, блокувальні зауваження окремо від довідкових, **чернетка відгуку** мовою оголошення. Кнопка «Скопіювати відгук».
6. Стійкий прийом відповіді	Нормалізовані цитати; неправильні висновки відкидаються поштучно; один авто-повтор; збій формату не входить у ліміт 4 запусків, але записується в журнал і в час.
7. Модель	Рекомендація за пам'яттю, запам'ятований вибір; `auto` більше не бере найменшу модель.
8. Автозаміри	Мітки часу: вставлено → розібрано → підтверджено → рішення. Кнопка «Експорт пілота в CSV» замість ручного заповнення `real-pilot-template.csv`.
9. Захист даних	`PRAGMA user_version`, автоматична копія бази перед міграцією.

> **Ціль етапу (перевірити заміром, а не вважати досягнутою):** одне оголошення від вставлення до рішення за ≤ 3 хв активного часу; ≥ 80% відповідей моделі приймаються з першої спроби; 0 випадків, коли бюджет виконавця показано як бюджет замовника.

Наявні 106 тестів мають лишитися зеленими. Нові тести: розбір без цитати → «невідомо»; поштучне відкидання висновків; повтор не з'їдає квоту; міграція схеми з бази 0.6.1.

Етап 2 · пілот із 10 задач на 0.7

Протокол беремо з `docs/Orchestrator-Real-Pilot-RU.md` і з плану ChatGPT: способи А (звичайний ChatGPT) і Б (програма) чергуються; критерії критичних помилок записуються до початку; повторне знайомство з матеріалом зазначається як обмеження. Різниця одна: вимірюємо на версії 0.7.

Метрики

МЕТРИКА	ЯК РАХУВАТИ	ПОРІГ РІШЕННЯ
Активний час	Пошук + вставлення + підтвердження + перевірка + правки. Очікування моделі окремо. Мітки ставить програма.	Медіана Б ≤ 0,8 × медіани А
Критичні помилки	Помилка, через яку людина ухвалила б хибне рішення про відгук: бюджет, строк, статус, обов'язкова вимога	0
Придатний результат	Заздалегідь задані 5 критеріїв (як у ChatGPT)	≥ 9 із 10, не гірше за А
Прийом з першої спроби	Відповіді моделі без повтору / усі відповіді	≥ 80%
Правки власника	Кількість полів, які власник змінив після розбору	≤ 1 у середньому на оголошення
Корисність чернетки	Чернетку відгуку надіслано без змін / з дрібними правками / переписано	≥ 50% без змін або з дрібними правками

Етап 3 · два тижні реального використання

- **Воронка:** розглянуто → відгукнувся → відповідь → виграно → години → дохід. Поле в картці, без окремого розділу.
- **Самостійне повернення:** скільки днів із 14 власник відкривав програму без нагадування. Ціль ≥ 5.
- Після 20–30 відгуків порівняти бал відповідності з тим, на які замовлення відповіли замовники, і переглянути ваги 0,5 / 0,3 / 0,2. Без цих даних ваги лишаються припущенням.

Рішення за результатами

ЩО ПОКАЗАЛИ ЗАМІРИ	НАСТУПНА ІТЕРАЦІЯ
Найдовше — знайти оголошення	Один адаптер лише для читання (RSS або поштові сповіщення біржі, у межах її правил) + дедуплікація, яка вже є (`history`) + щоденний дайджест
Найбільше правок після розбору	Кращі приклади формату, більша модель, перевіряльник лише для блокувальних полів
Збої й таймаути	Класи помилок і керований повтор (тут план ChatGPT стає актуальним)
Часу не менше, ніж у А	Залишити звичайний ChatGPT + ведення воронки; не розширювати програму

Спрощення і що поки не робити

Прибрати з головного шляху

- Розділи 04–06 перенести за перемикач «Експериментальне». Код не видаляти: він протестований.
- Документи: README, CHANGELOG, протокол пілота. Паспорт, бізнес-план автономності, план диференціації та звіти етапів перенести в `docs/archive/`.
- Одна мова інтерфейсу й документації. Зараз «Досвід і клієнти» українською, решта російською.
- Менше застережень в інтерфейсі: одне чітке «модель не перевіряє сторінки», а не по абзацу в кожному блоці.
- `app.py`: ланцюжок `if` замінити таблицею маршрутів. Це полегшить додавання ендпоінтів етапу 1.

Не робити до результатів етапу 2

- Повне приймання інструкцій на 60 оцінок і швидкий прогін на 16.
- Продаж звітів із таблиць, «автономну компанію», клієнтські брифи.
- Автоматичне надсилання відгуків або повідомлень замовникам.
- Детальний облік витрат на підписки: на рішення «чи користуватися» він не впливає.
- Нові ролі агентів. Бригаду (аналітик + перевіряльник) вмикати тільки для оголошень із блокувальними зауваженнями.

Шлях до автономності (стиснена драбина ChatGPT)

РІВЕНЬ	МОЖЛИВІСТЬ	ДОКАЗ ДЛЯ ПЕРЕХОДУ
A0	Поточна версія 0.6.1: ручний JSON	—
A1	0.7: вставлення → розбір → картка рішення → чернетка відгуку	Пілот: −20% часу, 0 критичних помилок
A2	Один адаптер лише для читання + щоденний дайджест	10 перевірених імпортів; правила джерела підтверджено; ≥ 5 днів самостійного використання з 14
A3	Виконання одного типового замовлення (наприклад, лендинг)	Воронка показує виграні замовлення; два оплачені пілоти
A4	Фонове повторення погоджених операцій	Межі доступу й витрат, журнал, зупинка, відкат

До профілю відбору варто додати те, що пропонує ChatGPT: мінімальний бюджет, географію, доступний час, підтверджені навички, виключені майданчики. Розбір моделлю тоді зможе одразу відсіювати очевидні невідповідності.

Порядок робіт

КОЛИ	ЩО	РЕЗУЛЬТАТ
День 0, 1 год	На своєму комп'ютері: CHECK → START → один реальний прогін на 0.6.1 із ручним ChatGPT. Засікти час заповнення JSON окремо від решти.	Базова точка: скільки хвилин іде на JSON. Це підтвердить або спростує головну тезу документа.
Дні 1–5	Етап 1: версія 0.7 «вставив → рішення» (с. 4)	Версія 0.7, тести зелені, один живий прогін
Тиждень 2	Етап 2: 10 задач, А/Б по черзі	Експорт CSV, рішення за порогами
Тижні 3–4	Етап 3: реальні відгуки, воронка	Дані для ваг і для вибору наступного вузького місця
Далі	Тільки під виміряне вузьке місце (с. 5)	A2 або спрощення

Спільне в обох планах. ChatGPT і я сходимося в головному: ефективність визначається часом і якістю на реальних задачах, а не кількістю тестів, агентів чи розділів. Різниця в тому, що я пропоную спершу прибрати відомий ручний крок і тоді вимірювати. Якщо прогін дня 0 покаже, що JSON займає небагато часу, правий ChatGPT, і пілот можна запускати одразу на 0.6.1.

Як прийняти рішення між двома планами

1. Зробити один прогін дня 0 (у будь-якому разі потрібен обом планам).
2. Якщо підготовка матеріалу займає **більше 40%** активного часу, починати з етапу 1 цього плану.
3. Якщо менше, запускати пілот за планом ChatGPT, а до етапу 1 повернутися за його результатами.

Підстави висновків і межі перевірки

ПЕРЕВІРКА	РЕЗУЛЬТАТ
Цілісність архіву	52 контрольні суми з `MANIFEST.json` збігаються.
Технічний прогін	106 тестів, 63,5 с, Linux, Python 3.13. Результат: OK.
Вердикт шаблону	`assess(validate_item(шаблон))` → `needs_review`, 6 зауважень: первинна сторінка, автор перевірки, дата, підтвердження прийому заявок, відповідність вимогам, обґрунтування.
Розмір контексту	5 оголошень із `pilot-materials.json` → 18 952 байти запиту; ≈ 2,6 КБ на оголошення, ≈ 6 КБ базового запиту; ліміт 32 000 → ≈ 9–10 оголошень.
Історичне демо	На поточну дату всі 5 демо-оголошень мають вердикт `excluded` (строки минули). Так і має бути, бо демо прив'язане до 25.09.2026.
Звіт і висновок агента	За кодом: `render_report` не використовує `agent_runs`. Збігається з пробою ChatGPT.
Часова нестабільність	За кодом: `assess()` → `projection` → `snapshot_hash`. Сам часовий прогін не повторював, механізм збігається з описом ChatGPT.
Вибір моделі	За кодом: `AgentService.start` бере `models[0]` зі списку, відсортованого за розміром.

Що не підтверджено

Живих викликів Ollama, Claude чи ChatGPT у цьому аналізі не було. Програму в браузері не запускав. Фізичні Mac/Windows, якість моделей, реальну економію часу та дохід не вимірював. Оцінки трудомісткості і пороги на с. 4–5 — пропозиції для пілота, а не досягнуті показники. Розбір тексту моделлю (етап 1) теж може помилятися, тому форма підтвердження критичних полів обов'язкова.

Де змінювати код

`core.py`: `validate_item`, `assess`, `rank_items`, `render_report`, `Store.__init__` (міграції).
`agents.py`: `SCHEMA`, `validate_result`, `start`, `MAX_RUNS_PER_TASK`, нова роль «розбір оголошення».
`providers.py`: `discover` (рекомендація моделі).
`app.py`: маршрути вставлення, картки рішення, експорту пілота.
`static/app.js`, `index.html`: екран вставлення, форма підтвердження, картка рішення; розділи 04–06 за перемикачем.

Джерела

1. Архів `Orchestrator-Pilot-0.6.1-Review-Fixes.zip`, код і документація 0.6.1.
2. `Orchestrator-Improvement-Plan-UA-2026-10-15.pdf` (план ChatGPT, 8 с.).
3. Власний прогін тестів і перевірки, перелічені вище, 15.10.2026. Код програми не змінювався.